МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 7

з дисципліни „ **Основи вбудованих систем**"

на тему „ Визначення рівня звуку за допомогою датчика звуку"

Виконав

студент 2 курсу

групи ФЕП-23

Рак Богдан Ігорович

Перевірив

доцент кафедри РКС

Павлик Михайло Романович

Львів – 2025

Untitled

```python
import asyncio
from machine import Pin, ADC, SoftI2C, PWM
from OLED_1inch5 import OLED_1inch5
from time import sleep
import utime

i2c = SoftI2C(scl=Pin(7), sda=Pin(6), freq=1_000_000)
sleep(0.1)
buzzer = PWM(Pin(12))
button = Pin(3, Pin.IN, Pin.PULL_UP)
sound_sensor = ADC(Pin(28))
OLED = OLED_1inch5(0x3d, i2c)

melody_str = "Melody:b=116,o=4,d=4:e,8b,8p,8b,8a,8b,e,8b,8p,8b,8p,8a,b,d5,8c5,8p,8c5,8b,8c5,e,8c5,8p,8c5,8p,8b,c5,d5"

NOTES = {
    'c': 261.63, 'c#': 277.18, 'd': 293.66, 'd#': 311.13, 'e': 329.63,
    'f': 349.23, 'f#': 369.99, 'g': 392.00, 'g#': 415.30, 'a': 440.00,
    'a#': 466.16, 'b': 493.88, 'p': 0
}

def parse_rtttl(rtttl_str):
    parts = rtttl_str.split(':')
    header = parts[1].split(',')
    notes_data = parts[2].split(',')
    bpm = 116
    default_duration = 4
    default_octave = 4
    for h in header:
        if h.startswith('b='): bpm = int(h[2:])
        if h.startswith('d='): default_duration = int(h[2:])
        if h.startswith('o='): default_octave = int(h[2:])
    wholenote = (60 * 1000 / bpm) * 4
    parsed_notes = []
    for n in notes_data:
        n = n.strip()
        if not n: continue
        i = 0
        duration = ""
        while i < len(n) and n[i].isdigit():
            duration += n[i]
            i += 1
        note_duration = int(duration) if duration else default_duration
        note_name = ""
        while i < len(n) and not n[i].isdigit():
            note_name += n[i]
            i += 1
        octave = int(n[i:]) if i < len(n) else default_octave
        freq = 0
        if note_name in NOTES:
            freq = NOTES[note_name]
            if freq > 0:
                freq *= (2 ** (octave - 4))
        ms_duration = wholenote / note_duration
        parsed_notes.append((freq, ms_duration))
    return parsed_notes

melody_notes = parse_rtttl(melody_str)

state = {
    "running":       True,
    "sound_value":   0,
    "column_hights": [],
}

async def task_button():
    prev = 1
    while True:
        cur = button.value()
        if prev == 1 and cur == 0:
            state["running"] = not state["running"]
            if not state["running"]:
                buzzer.duty_u16(0)
                state["column_hights"].clear()
                OLED.fill(0)
                OLED.text("  PAUSED  ", 14, 56, 15)
                OLED.show()
        prev = cur
        await asyncio.sleep_ms(50)
```

Untitled

```
async def task_adc():
    while True:
        if state["running"]:
            vol = sound_sensor.read_u16()
            state["sound_value"] = vol / 65535 * 100
        await asyncio.sleep_ms(100)

async def task_melody():
    while True:
        if not state["running"]:
            await asyncio.sleep_ms(50)
            continue
        for freq, ms_duration in melody_notes:
            while not state["running"]:
                await asyncio.sleep_ms(50)
            active_ms = int(ms_duration * 0.9)
            pause_ms  = int(ms_duration * 0.1)
            if freq > 0:
                buzzer.freq(int(freq))
                buzzer.duty_u16(30000)
            else:
                buzzer.duty_u16(0)
            await asyncio.sleep_ms(active_ms)
            buzzer.duty_u16(0)
            await asyncio.sleep_ms(pause_ms)

async def task_display():
    while True:
        if not state["running"]:
            await asyncio.sleep_ms(50)
            continue
        sound = state["sound_value"]
        col = state["column_hights"]
        col.append(int(sound))
        if len(col) > 64:
            del col[0]
        OLED.fill(0)
        OLED.text("Sound={0:.1f}%".format(sound), 1, 5, 15)
        i = 0
        for ch in col:
            if ch > 0:
                OLED.fill_rect(i, 127 - ch, 2, ch, 15)
            i += 2
        OLED.show()
        await asyncio.sleep_ms(100)

async def main():
    await asyncio.gather(
        task_button(),
        task_adc(),
        task_melody(),
        task_display(),
    )

asyncio.run(main())
```

Висновок: Виконуючи цю лабораторну роботу, можна навчитись визначати рівень звуку за допомогою датчика звуку та виводити інформацію про рівень звуку на OLED дисплей.